

Fitly: Design Patterns and SOLID Principles

Social Fitness App MVP | Team: Paxton, Paulo, Raymond

Git branch: **design-patterns-solid**

<https://github.com/paulopsss/social-fitness-app/tree/design-patterns-solid>

This report applies design patterns and SOLID principles to the Fitly backend (Java 17, Spring Boot). Section 1 picks four patterns that fit the app and shows each one in UML. Section 2 implements the Observer pattern with before and after code for the Log Workout use case. Section 3 reviews the existing login code against all five SOLID principles, finds three violations, and refactors them. Section 4 shows how the packages are organized. All code is in the Git branch linked above, and the code listings in this report are copied from it (package and import lines are left out to save space).

What is in the branch

Item	Where to look
Observer pattern (Section 2)	service/WorkoutService, WorkoutObserver, GoalProgressObserver, NotificationObserver; model/Workout; docs/before/WorkoutServiceBefore.java.txt
SOLID refactor (Section 3), which also puts Repository and Strategy into the login code	web/AuthController, LoginSession, Pages, SessionAuthFilter; service/AuthService; security/PasswordHasher, Pbkdf2PasswordHasher; repository/UserRepository, InMemoryUserRepository; model/User
Original code before the refactor	docs/before/ (AuthController, UserStore, SessionAuthFilter)
Tests	src/test/java, six test classes, run with mvn test or Run All Tests in IntelliJ
This report	docs/Fitly_Design_Patterns_SOLID.pdf

1. Design patterns that fit Fitly

Each pattern below solves a real problem in this project. Repository and Strategy are put into the working login code in Section 3. Observer is implemented in Section 2. Chain of Responsibility is already in the code through the servlet filter.

Pattern	Problem it solves in Fitly	Code
Repository	Login code is tied to an in-memory map, but ADR 5 moves us to H2	repository/UserRepository
Strategy	Passwords were compared as plain text, and Use Case 1 says they must be stored securely	security/PasswordHasher
Observer	Logging a workout must update goals and send notifications, and later feed analytics	service/WorkoutObserver
Chain of Responsibility	Pages need separate checks: login now, admin later	web/SessionAuthFilter

1.1 Repository

Problem in Fitly. The old UserStore was a map that the controller used directly. ADR 5 in our architecture document says the map will be replaced by an H2 database. Without a clear boundary, that change would reach into the login code.

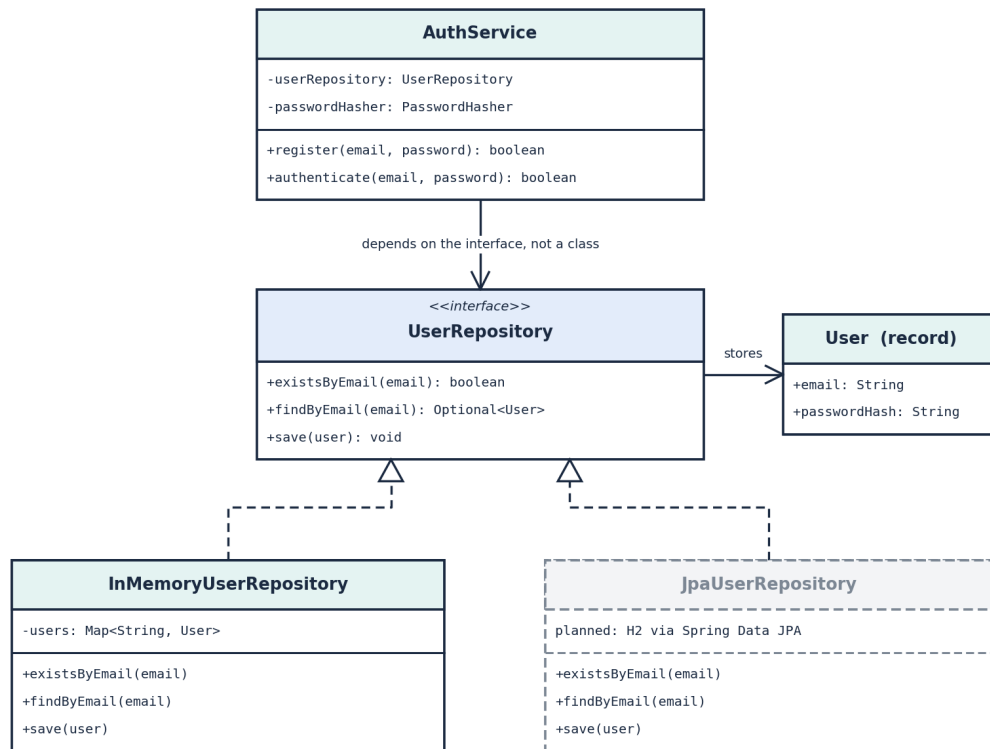


Figure 1: Repository pattern applied to Fitly. The dashed box is the planned H2 version.

How it applies. AuthService asks a UserRepository interface for users. InMemoryUserRepository is what runs today. A JpaUserRepository for H2 can take its place later and AuthService does not change.

Why this is the right solution.

- We already know the storage will change, so the change should cost one new class, not edits across the login flow.
- Tests use the fast in-memory version, so they need no database.
- Spring Data JPA repositories follow this same idea, so the team will meet it again when H2 is added instead of learning something new.
- Other options were calling a database class straight from the service (simple now, same rewrite later) and a full DAO layer with mapping objects (more than an MVP needs).

Trade-off. One extra interface for a very small amount of data.

1.2 Strategy

Problem in Fitly. UserStore compared passwords with equals(), so they were stored as plain text. Use Case 1 says the password must be stored securely. Good hashing settings also change over time as computers get faster.

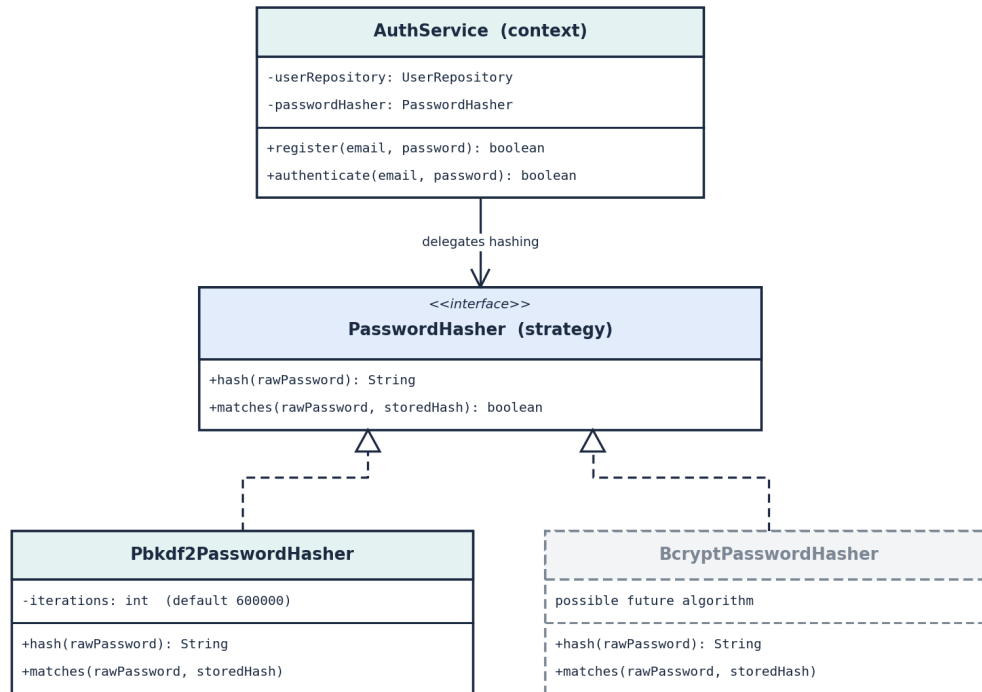


Figure 2: Strategy pattern applied to password hashing.

How it applies. PasswordHasher has two methods, hash() and matches(). Pbkdf2PasswordHasher is the strategy used today. AuthService only knows the interface. Each stored hash includes its own iteration count, so old hashes keep working when the setting is raised.

Why this is the right solution.

- Choosing an algorithm is a separate decision from the login flow and changes for different reasons. A strategy lets us add BCrypt or Argon2 with one new class.
- Tests use the same class with 1,000 iterations instead of 600,000, so they run in milliseconds.
- The alternative, an if/else inside AuthService that reads a setting, would mean editing and retesting the login code for every new algorithm.

Trade-off. Today there is one real strategy, so the interface is a small cost paid early. We accept it because the security requirement makes change likely.

1.3 Observer

Problem in Fitly. Use Case 2 says that logging a workout updates goal progress and creates a notification. The usage analytics feature will want the same event later. If WorkoutService calls each of these directly, it has to know about all of them.

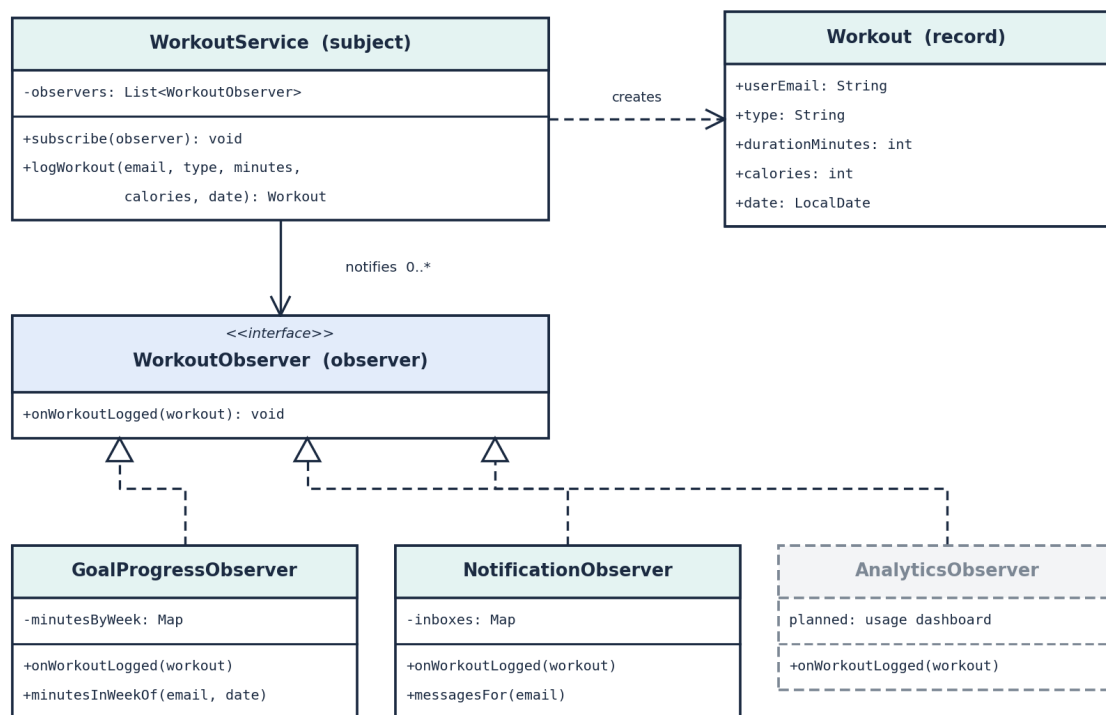


Figure 3: Observer pattern applied to Log Workout. The dashed box is a planned observer.

How it applies. `WorkoutService` is the subject. It keeps a list of `WorkoutObserver` objects and tells each one when a workout is logged. `GoalProgressObserver` and `NotificationObserver` are the first two observers. An `AnalyticsObserver` can be added later.

Why this is the right solution.

- Three teammates will work on goals, notifications, and analytics at the same time. With observers, each person owns one class, so there are fewer merge conflicts in `WorkoutService`.
- A new reaction is a new class. `WorkoutService` and its tests stay the same.
- `WorkoutService` can be tested with tiny fake observers.
- Spring's event publisher would also work. The hand-written version makes the pattern visible and is easy to test without Spring. A message queue is far too much for one app.

Trade-off. The flow is less obvious when reading the code. Observers run one after another in the same thread, and an exception in one stops the ones after it. Before production we would catch and log errors per observer.

1.4 Chain of Responsibility

Problem in Fitly. Some pages need login. The use case diagram also has Moderate Users/Content for admins, which will need an admin check. Putting every check in one place would make one large method that is hard to test.

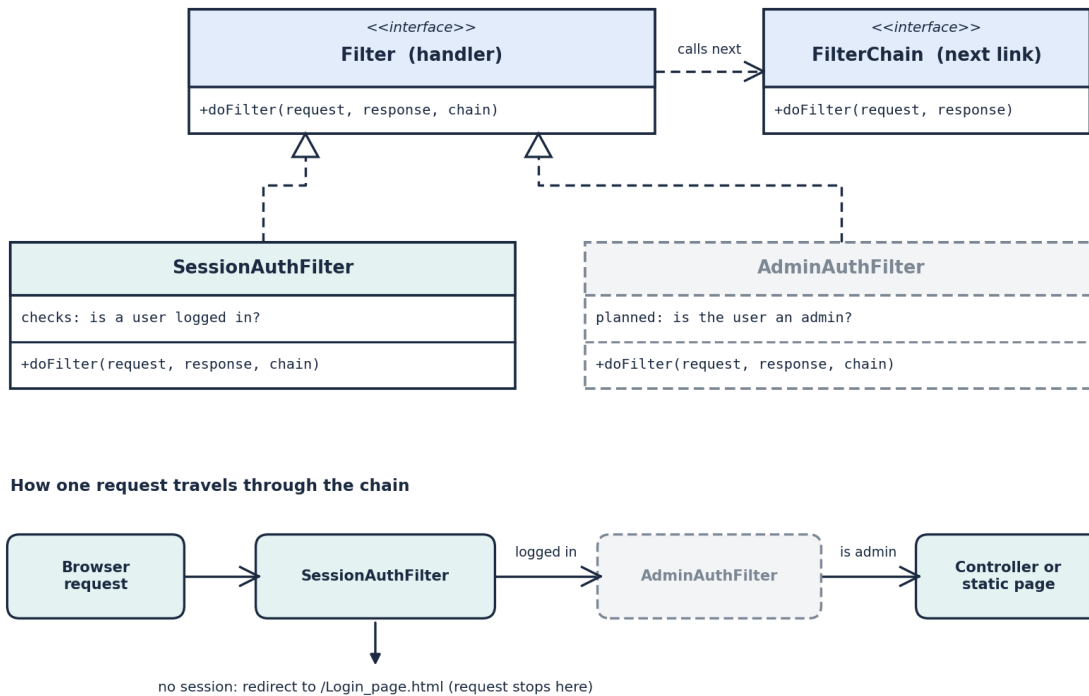


Figure 4: Chain of Responsibility in the filter chain, with the planned admin filter.

How it applies. Servlet filters already form a chain. `SessionAuthFilter` is one link: it either stops the request with a redirect to login or passes it on. A future `AdminAuthFilter` would be the next link, registered after it.

Why this is the right solution.

- Each link has one job, so each is small and easy to test.
- Adding the admin check means adding a class and registering it in `SecurityConfig`. `SessionAuthFilter` is not touched.
- The order is explicit: log in first, then check the role.
- The servlet container already supports the pattern, so we write no chain code.

Trade-off. Order matters, and forgetting to register a filter silently skips a check. `ProtectedPageTest` guards the login check.

2. Pattern implementation: Observer for Log Workout

Use Case 2 (Log Workout) and its sequence diagram say that after a workout is created, goal progress is updated and a notification is created. The first listing shows the direct way to write this. The second shows the Observer version that is in the branch.

Before: without the pattern

BEFORE (docs/before/WorkoutServiceBefore.java.txt)

```
public class WorkoutService {

    // The service must know every follow-up step by name.
    private final GoalService goalService;
    private final NotificationService notificationService;
    // Adding analytics later means: a new field, a new constructor argument,
    // a new call below, and an edit to every test that builds this class.

    public WorkoutService(GoalService goalService, NotificationService notificationService) {
        this.goalService = goalService;
        this.notificationService = notificationService;
    }

    public Workout logWorkout(String userEmail, String type, int durationMinutes,
                             int calories, LocalDate date) {
        Workout workout = new Workout(userEmail, type, durationMinutes, calories, date);

        // Hard wired, in a fixed order. Whoever owns goals and whoever owns
        // notifications both have to edit this same method.
        goalService.addProgress(userEmail, date, durationMinutes);
        notificationService.send(userEmail, "Workout logged: " + type + ", " + durationMinutes + " min");

        return workout;
    }
}

// Collaborators the service above depends on directly:
// class GoalService      { void addProgress(String userEmail, LocalDate date, int minutes) { ... } }
// class NotificationService { void send(String userEmail, String message) { ... } }
```

What is wrong with this version

- WorkoutService has to know every follow-up step by name.
- Adding analytics means a new field, a new constructor argument, a new call, and edits to every test that builds the service.
- Whoever builds goals and whoever builds notifications both edit the same method, so merge conflicts are likely.
- Even a test about saving a workout needs a GoalService and a NotificationService, or mocks of both.

After: with the pattern

The interface is one method. Anything that wants to react to a workout implements it.

AFTER service/WorkoutObserver.java

```
/**
 * Observer pattern: anything that needs to react when a workout is logged
 * implements this one method.
 *
 * Observers must not depend on each other or on running in a set order.
 */
public interface WorkoutObserver {

    void onWorkoutLogged(Workout workout);
}
```

WorkoutService no longer knows who is listening. Spring passes in every WorkoutObserver bean, and subscribe() adds more at run time.

AFTER service/WorkoutService.java

```
/**
 * The subject in the Observer pattern (Use Case 2: Log Workout).
 *
 * IMPROVEMENT over the "before" version: this class no longer knows who
 * reacts to a workout. To add a reaction, for example analytics, write a new
 * WorkoutObserver. This class, and its tests, do not change.
 *
 * Spring finds every WorkoutObserver bean and passes the list in, so there
 * is no registration code to write. subscribe() is the classic Observer
 * "attach" operation and is handy in tests.
 */
@Service
public class WorkoutService {

    private final List<WorkoutObserver> observers;

    public WorkoutService(List<WorkoutObserver> observers) {
        this.observers = new CopyOnWriteArrayList<>(observers);
    }

    public void subscribe(WorkoutObserver observer) {
        observers.add(observer);
    }

    public Workout logWorkout(String userEmail, String type, int durationMinutes,
                             int calories, LocalDate date) {
        Workout workout = new Workout(userEmail, type, durationMinutes, calories, date);
        // Saving the workout goes here once the H2 database is added.
        for (WorkoutObserver observer : observers) {
            observer.onWorkoutLogged(workout);
        }
        return workout;
    }
}
```

The two observers hold the logic that used to live inside the direct calls.

AFTER service/GoalProgressObserver.java

```
/**
 * Reacts to a workout by adding its minutes to that user's total for the
 * week the workout happened in (weeks follow the ISO calendar, Monday first).
 * Comparing this total with a WeeklyGoal target comes with the goals feature.
 */
@Component
public class GoalProgressObserver implements WorkoutObserver {

    private final Map<WeekKey, Integer> minutesByWeek = new ConcurrentHashMap<>();

    @Override
    public void onWorkoutLogged(Workout workout) {
        minutesByWeek.merge(WeekKey.of(workout.userEmail(), workout.date()),
            workout.durationMinutes(), Integer::sum);
    }

    /** Total minutes the user logged in the week that contains the given date. */
    public int minutesInWeekOf(String userEmail, LocalDate dateInWeek) {
        return minutesByWeek.getOrDefault(WeekKey.of(userEmail, dateInWeek), 0);
    }

    private record WeekKey(String userEmail, int weekBasedYear, int week) {
        static WeekKey of(String userEmail, LocalDate date) {
            return new WeekKey(userEmail,
                date.get(IsoFields.WEEK_BASED_YEAR),
                date.get(IsoFields.WEEK_OF_WEEK_BASED_YEAR));
        }
    }
}
```

AFTER service/NotificationObserver.java

```
/** Reacts to a workout by creating a confirmation message for that user. */
@Component
public class NotificationObserver implements WorkoutObserver {

    private final Map<String, List<String>> inboxes = new ConcurrentHashMap<>();

    @Override
    public void onWorkoutLogged(Workout workout) {
        String message = "Workout logged: " + workout.type()
            + ", " + workout.durationMinutes() + " min";
        inboxes.computeIfAbsent(workout.userEmail(), email -> new CopyOnWriteArrayList<>())
            .add(message);
    }

    public List<String> messagesFor(String userEmail) {
        return List.copyOf(inboxes.getOrDefault(userEmail, List.of()));
    }
}
```

What improved

Change	Before	After
Add analytics	Edit WorkoutService, its constructor, and its tests	Add one class that implements WorkoutObserver
Test WorkoutService	Needs a goal service and a notification service	Pass a list of small fake observers
Teammates working in parallel	All edit the same method	Each person owns one observer class
Order of reactions	Fixed in the method body	Observers must not depend on order (stated in the interface comment)

How Spring wires it

WorkoutService takes a `List<WorkoutObserver>` in its constructor. Spring finds every bean that implements `WorkoutObserver` and passes the list in, so there is no registration code. `WorkoutWiringTest` checks that this works. The `subscribe()` method is the classic Observer attach operation and is used in the unit tests.

Tests for this pattern

- **WorkoutServiceTest:** every observer is told about a logged workout; an observer added later is told about later workouts; an invalid workout is rejected and nobody is told; goal minutes add up per user and per ISO week; the notification message goes to the right user.
- **WorkoutWiringTest:** Spring injects both observers into WorkoutService.

3. SOLID analysis of the existing login code

The code reviewed is the login code from the add-backend branch: AuthController, UserStore, SessionAuthFilter, and SecurityConfig. The original files are saved in docs/before/ so the before and after can be compared.

Principle	Verdict	Evidence in the original code
S: Single Responsibility	Violated	AuthController handled HTTP, the rule "no duplicate email, then log in", and how login is stored in the session. UserStore stored users and also decided how passwords are compared.
O: Open/Closed	Violated	Changing how passwords are checked (to add hashing) meant editing UserStore.checkCredentials.
L: Liskov Substitution	No violation	The only inheritance is SessionAuthFilter implementing the servlet Filter interface. It follows the contract: it either completes the response or passes the request on. The new interfaces state their contracts too, for example matches() returns false instead of throwing for an unreadable hash.
I: Interface Segregation	No violation	The original code defined no interfaces of its own, and SessionAuthFilter implements only doFilter. The new interfaces are small (3, 2, and 1 methods) and every method is used by its client.
D: Dependency Inversion	Violated	AuthController depended on the concrete UserStore class, so the login flow was tied to an in-memory map.

3.1 Violation: Single Responsibility (AuthController)

AuthController had three reasons to change: the web layer (form fields and redirects), the registration rule, and the way login state is stored. The filter also repeated the session key and the login path, so two classes had to be kept in sync by hand.

BEFORE AuthController.register() and SessionAuthFilter (original)

```
@PostMapping("/register")
public String register(@RequestParam String email,
                      @RequestParam String password,
                      HttpSession session) {
    // SRP violation: a business rule (no duplicate emails, then log in) sits in the controller.
    if (userStore.exists(email)) {
        return "redirect:/Register_page.html?error=1";
    }
    userStore.addUser(email, password);
    session.setAttribute("user", email);
    return "redirect:/Main_page.html";
}

// ... and in SessionAuthFilter:
// Duplicated knowledge: the key "user" and the path must match AuthController by hand.
boolean loggedIn = session != null && session.getAttribute("user") != null;
```

AFTER web/AuthController.java (only translates between HTTP and the service)

```
/**
 * Only translates between HTTP and the AuthService: read the form, ask the
 * service, start or end the session, pick the redirect. The rules live in
 * AuthService (Single Responsibility).
 */
@Controller
public class AuthController {

    private final AuthService authService;

    public AuthController(AuthService authService) {
        this.authService = authService;
    }

    @PostMapping("/login")
    public String login(@RequestParam String email,
                       @RequestParam String password,
```

```

        HttpSession session) {
    if (authService.authenticate(email, password)) {
        LoginSession.start(session, email);
        return "redirect:" + Pages.DASHBOARD;
    }
    return "redirect:" + Pages.LOGIN + Pages.ERROR_QUERY;
}

@PostMapping("/register")
public String register(@RequestParam String email,
    @RequestParam String password,
    HttpSession session) {
    if (!authService.register(email, password)) {
        return "redirect:" + Pages.REGISTER + Pages.ERROR_QUERY;
    }
    LoginSession.start(session, email);
    return "redirect:" + Pages.DASHBOARD;
}

@PostMapping("/logout")
public String logout(HttpSession session) {
    LoginSession.end(session);
    return "redirect:" + Pages.HOME;
}
}

```

AFTER service/AuthService.java (the rules, with no HTTP)

```

/**
 * The login and registration rules, with no HTTP in sight (Single Responsibility).
 *
 * Both collaborators are interfaces (Dependency Inversion), so this class can
 * be tested with plain Java and works unchanged when storage moves to H2 or
 * the hashing algorithm changes.
 */
@Service
public class AuthService {

    private final UserRepository userRepository;
    private final PasswordHasher passwordHasher;

    public AuthService(UserRepository userRepository, PasswordHasher passwordHasher) {
        this.userRepository = userRepository;
        this.passwordHasher = passwordHasher;
    }

    /** @return true if the account was created, false if the email is already taken */
    public boolean register(String email, String password) {
        if (userRepository.existsByEmail(email)) {
            return false;
        }
        userRepository.save(new User(email, passwordHasher.hash(password)));
        return true;
    }

    /** @return true only if the email exists and the password matches */
    public boolean authenticate(String email, String password) {
        return userRepository.findByEmail(email)
            .map(user -> passwordHasher.matches(password, user.passwordHash()))
            .orElse(false);
    }
}

```

AFTER web/LoginSession.java (the one place that knows how login is stored)

```

/**
 * The one place that knows how "logged in" is stored in the session.
 * Before this class existed, the controller and the filter each repeated
 * the string "user", and they had to stay in sync by hand.
 */
public final class LoginSession {

    private static final String USER_KEY = "user";

    private LoginSession() {
    }
}

```

```

    public static void start(HttpSession session, String email) {
        session.setAttribute(USER_KEY, email);
    }

    /** The session can be null when the visitor has never had one. */
    public static boolean isActive(HttpSession session) {
        return session != null && session.getAttribute(USER_KEY) != null;
    }

    public static void end(HttpSession session) {
        session.invalidate();
    }
}

```

AFTER web/SessionAuthFilter.java (the check now goes through LoginSession)

```

    if (!LoginSession.isActive(req.getSession(false))) {
        res.sendRedirect(Pages.LOGIN);
        return;
    }

```

Why this is better

- Each class has one reason to change: web details, registration rules, or session storage.
- AuthService is tested with plain Java and no web server (AuthServiceTest).
- The session key lives only in LoginSession and the page paths only in Pages, so the filter and controller cannot drift apart.

3.2 Violation: Open/Closed (UserStore password check)

To store passwords securely, the old code had to be edited in place. Every change to the password rule meant reopening and retesting a class that also handled storage. The fix is the Strategy pattern from Section 1.2.

BEFORE UserStore.checkCredentials() (original)

```

// SRP + OCP violation: this class stores users AND decides how passwords are compared.
// To add hashing, this method has to be edited.
public boolean checkCredentials(String email, String password) {
    String stored = users.get(email);
    return stored != null && stored.equals(password);
}

```

AFTER security/PasswordHasher.java (the strategy interface)

```

/**
 * Strategy pattern + Open/Closed Principle.
 *
 * How a password is turned into a stored value is a decision that changes
 * over time. AuthService only knows this interface, so a new algorithm is a
 * new class and AuthService stays untouched.
 *
 * Contract (every implementation must follow it):
 * - hash() never returns the raw password.
 * - matches() returns false, and does not throw, for a wrong password or a
 *   stored value it cannot understand.
 */
public interface PasswordHasher {

    String hash(String rawPassword);

    boolean matches(String rawPassword, String storedHash);
}

```

AFTER security/Pbkdf2PasswordHasher.java (the strategy used today, key parts)

```
@Override
public String hash(String rawPassword) {
    byte[] salt = new byte[SALT_BYTES];
    random.nextBytes(salt);
    byte[] key = derive(rawPassword, salt, iterations);
    return PREFIX + "$" + iterations + "$" + encode(salt) + "$" + encode(key);
}

@Override
public boolean matches(String rawPassword, String storedHash) {
    if (rawPassword == null || storedHash == null) {
        return false;
    }
    String[] parts = storedHash.split("\\$");
    if (parts.length != 4 || !PREFIX.equals(parts[0])) {
        return false;
    }
    try {
        int storedIterations = Integer.parseInt(parts[1]);
        byte[] salt = Base64.getDecoder().decode(parts[2]);
        byte[] expected = Base64.getDecoder().decode(parts[3]);
        byte[] actual = derive(rawPassword, salt, storedIterations);
        return MessageDigest.isEqual(expected, actual);
    } catch (IllegalArgumentException e) {
        // Bad number, bad Base64, or an unusable salt: not a match.
        return false;
    }
}
```

Why this is better

- A new algorithm is a new class. AuthService is closed to the change and is not edited.
- Old hashes keep working because the iteration count is stored inside each hash (Pbkdf2PasswordHasherTest checks this).
- Passwords are no longer stored as plain text. Unreadable stored values return false instead of crashing the login.

3.3 Violation: Dependency Inversion (AuthController depends on UserStore)

The login flow, which is the important part, depended on a storage detail. Moving to H2 would have meant editing the controller and the store together. The fix is the Repository pattern from Section 1.1.

BEFORE AuthController (original)

```
// DIP violation: depends on the concrete UserStore class.
private final UserStore userStore;

public AuthController(UserStore userStore) {
    this.userStore = userStore;
}
```

AFTER repository/UserRepository.java (the abstraction)

```
/**
 * Repository pattern + Dependency Inversion.
 *
 * The login code talks to this interface and never to a map or a database.
 * Today InMemoryUserRepository implements it. When the team moves to H2
 * (ADR 5), a JPA based class can implement it instead and nothing above
 * this layer has to change.
 *
 * The interface is deliberately small: every method is used by AuthService.
 */
public interface UserRepository {

    boolean existsByEmail(String email);

    Optional<User> findByEmail(String email);

    void save(User user);
}
```

AFTER repository/InMemoryUserRepository.java (today's detail)

```
/**
 * Temporary storage: users vanish when the app restarts.
 * TODO: replace with an H2 backed implementation of UserRepository.
 * TODO: with a database, enforce one account per email using a unique
 * constraint, so existsByEmail followed by save cannot race.
 */
@Repository
public class InMemoryUserRepository implements UserRepository {

    private final Map<String, User> users = new ConcurrentHashMap<>();

    @Override
    public boolean existsByEmail(String email) {
        return users.containsKey(email);
    }

    @Override
    public Optional<User> findByEmail(String email) {
        return Optional.ofNullable(users.get(email));
    }

    @Override
    public void save(User user) {
        users.put(user.email(), user);
    }
}
```

AuthService (shown in 3.1) now takes a UserRepository and a PasswordHasher in its constructor, so both of its dependencies are interfaces.

Why this is better

- The rules in AuthService no longer know whether users live in a map or in H2. The detail depends on the abstraction, not the other way around.
- Switching to H2 means writing one JpaUserRepository class.
- AuthServiceTest builds the service with the in-memory class, so tests need no database.

3.4 Summary of the refactor

Original	Now
UserStore	repository/UserRepository + InMemoryUserRepository (storage), security/PasswordHasher + Pbkdf2PasswordHasher (password rule), service/AuthService (login and registration rules)
AuthController (rules and session)	web/AuthController (HTTP only) + web/LoginSession (session) + web/Pages (paths)
SessionAuthFilter and SecurityConfig	Same behavior, now using LoginSession and Pages; SecurityConfig moved to the config package

The URLs and redirects are unchanged, so the existing ProtectedPageTest did not need to change. The one visible difference is that passwords are now stored as hashes, which also closes the plain text TODO from the original code.

4. Code organization

The code is split into packages by job, in layers. Requests flow down from the browser, and each layer only uses the layer below it.

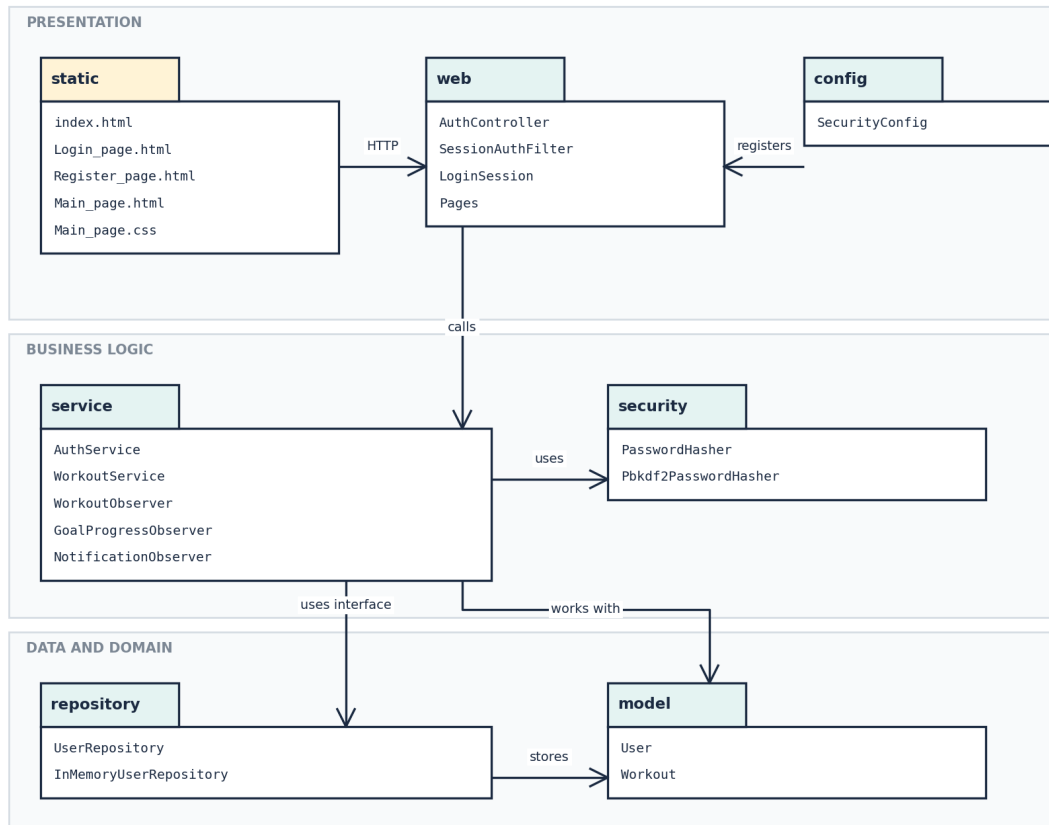


Figure 5: Package structure. Arrows show which package uses which.

Package	Responsibility	Uses
static (HTML, CSS)	What the browser shows. No Java.	Talks to web over HTTP
web	HTTP only: form fields, redirects, the session, the login filter	service
config	Spring wiring, such as which pages need login	web
service	Business rules: login, registration, logging a workout	repository, security, model
security	Password hashing	Java standard library only
repository	Storing and finding data	model
model	Plain data objects (User, Workout)	nothing

How this supports separation of concerns

- **One direction of dependencies.** web calls service, service uses repository and security, and everything may use model. Nothing calls back up, so a change low in the stack does not ripple upward.
- **HTTP stays in one package.** Only web and config import servlet or Spring web classes. The service, security, repository, and model packages import none (checked against the import lines).
- **Details sit behind interfaces.** Storage (UserRepository) and password hashing (PasswordHasher) can be replaced by H2 or BCrypt without touching web or service.

- **Front end and back end are separate.** The static folder holds only HTML and CSS, so page work does not touch Java and Java work does not touch pages.
- **Tests follow the packages.** Service, security, and repository tests are plain Java with no web server. Only `ProtectedPageTest` and `WorkoutWiringTest` start Spring.
- **Easy to find things for a three person team.** Someone adding the goals feature knows to add a rule to service, a data class to model, and storage to repository.

5. Running the code

- Open the repository folder (the one containing `pom.xml`) in IntelliJ and let Maven finish loading.
- Run `FitlyApplication` and visit `http://localhost:8080`, or run all tests with `mvn test`.
- To see the real before and after for the login code, compare the branch with the `add-backend` branch, or open the files in `docs/before/`.